



Portfolio

Internationale Hochschule Fernstudium
Studiengang Master of Science – Informatik

MedicalTracker: Konzeption, Softwarearchitektur und Implementierung eines digitalen Assistenzsystems für den Pflegealltag

Modul:	Projekt: Software Engineering (DLMCSPSE01_D)
Tutor:	Prof. Dr. Dirk Simon
vorgelegt von:	Steffen Niklas Oehler
Datum:	09.09.2026
Matrikelnummer:	IU14142306
E-Mail:	steffen-niklas.oehler@iu-study.org

Inhaltsverzeichnis

Inhaltsverzeichnis	II
Abbildungsverzeichnis	IV
Tabellenverzeichnis	V
Glossar	VI
1 Projektdokumentation	1
1.1 Projektübersicht.....	1
1.2 Aufbau der Arbeit.....	1
1.3 Abgrenzung des Systembereichs	2
1.4 Entwicklungsmethodik	3
1.4.1 Vorgehensmodell	3
1.4.2 Methodik	3
1.5 Technologie & Systemarchitektur	4
1.6 Risikomanagement.....	5
1.7 Zeitplanung.....	6
2 Anforderungsdokumentation	8
2.1 Stakeholder-Analyse	8
2.2 Funktionale Anforderungen	9
2.3 UML-Anwendungsfalldiagramm.....	12
2.4 Nichtfunktionale Anforderungen.....	13
2.5 Datenschutz- und Sicherheitsanalyse.....	14
2.5.1 Einordnung nach DSGVO	14
2.5.2 Technische und organisatorische Schutzmaßnahmen	15
2.5.3 Anforderungen an einen potenziellen klinischen Realeinsatz	16
3 Spezifikationsdokumentation	17
3.1 Objektorientiertes Datenmodell & Geschäftsobjekte	17
3.2 Geschäftsprozesse.....	19
3.3 Geschäftsregeln	22
3.4 Schnittstellen & Validierung	24
3.4.1 Systemschnittstellen	24

3.4.2 Benutzerschnittstellen & UI-Konzept.....	26
3.4.3 Mehrstufiges Validierungskonzept.....	28
4 Architekturdokumentation	29
4.1 Technologieübersicht	29
4.2 Architekturübersicht.....	30
4.3 Komponenten- und Klassenstruktur.....	32
4.4 Dynamisches Systemverhalten (Verhaltenssicht – Kernprozess GP-03)	33
Literaturverzeichnis.....	36

Abbildungsverzeichnis

Abbildung 1 Anwendungsfalldiagramm	12
Abbildung 2 Klassendiagramm.....	19
Abbildung 3 Aktivitätsdiagramm	21
Abbildung 4 Verwaltungsdashboard.....	26
Abbildung 5 Patientensicht.....	27
Abbildung 6 Detailliertes Klassendiagramm des Gesamtsystems	32
Abbildung 7 UML-Sequenzdiagramm des Kernprozesses GP-03	34

Tabellenverzeichnis

Tabelle 1 Risikoanalyse	5
Tabelle 2 Zeitplanung	6
Tabelle 3 Stakeholder-Analyse	8
Tabelle 4 Nichtfunktionale Anforderungen	13
Tabelle 5 MVC Zuordnungsmatrix.....	31

Glossar

Begriff	Definition
Compliance (Medizin)	Die Zuverlässigkeit und Einhaltung der verordneten medizinischen Behandlung (z. B. pünktliche Einnahme von Medikamenten) durch die zu pflegende Person.
Flask	Ein leichtgewichtiges WSGI-Webanwendungs-Framework in Python, das nach dem Micro-Framework-Ansatz aufgebaut ist.
Jinja2	Eine weit verbreitete Template-Engine für Python, die zur dynamischen Generierung von HTML-Seiten im Flask-Backend genutzt wird.
MVP (Minimum Viable Product)	Das minimal funktionsfähige Produkt, das die Kernfunktionen (Medikation & Vitaldaten) erfüllt, um frühzeitig evaluierbar zu sein.
ORM (Object-Relational-Mapping)	Eine Technik der Softwareentwicklung, die Daten zwischen einer relationalen Datenbank (SQLite) und objektorientierten Programmiersprachen (Python) konvertiert (hier umgesetzt via SQLAlchemy).
Vitaldaten	Tägliche, messbare Körperfunktionen (z. B. Blutdruck, Puls, Körpergewicht, Blutzucker), die Aufschluss über den Gesundheitszustand liefern.

1 Projektdokumentation

Das erste Kapitel definiert den organisatorischen und methodischen Rahmen für die Entwicklung der Anwendung MedicalTracker. Nach der Problemabgrenzung und der Darstellung des Kundennutzens begründet die Arbeit die Wahl der Systemarchitektur (Python, Flask, SQLite). Anschließend werden die Projektrisiken systematisch analysiert sowie Zeitplan und Arbeitspakete strukturiert dargestellt.

1.1 Projektübersicht

Die häusliche Pflege von Angehörigen stellt für viele Menschen eine erhebliche organisatorische und emotionale Herausforderung dar. Die Koordination von Medikationsplänen, das regelmäßige Erfassen von Vitaldaten sowie die Vorbereitung auf Arzttermine erfolgen in der Praxis häufig noch auf Papier oder in unstrukturierten Notizen. Dies erhöht das Risiko von Fehldosierungen, vergessenen Einnahmen und Informationsverlusten bei medizinischen Konsultationen.

Das Projekt adressiert diese Problematik durch die Konzeption und Implementierung einer intuitiven, lokalen Webanwendung. MedicalTracker dient als zentrales digitales Assistenzsystem für den Pflegealltag. Die Anwendung verfolgt das Ziel, pflegende Angehörige bei der täglichen Dokumentation und Planung zu entlasten und zu pflegenden Personen durch eine barrierefreie Tagesansicht mehr Selbstständigkeit zu ermöglichen. Aus dieser Zielsetzung leitet sich die zentrale Projekt- und Forschungsfrage der Arbeit ab:

„Wie muss eine lokale, leichtgewichtige Webanwendung architektonisch konzipiert sein, um pflegende Angehörige bei der Medikations- und Vitaldatenverwaltung zu entlasten? Wie lässt sich dabei eine barrierefreie Nutzung für pflegebedürftige Personen realisieren, ohne die rechtliche Komplexität eines zertifizierten Medizinprodukts zu erzeugen?“

Der vollständige Quellcode der Webanwendung, sämtliche Konfigurationsdateien, automatisierte Testskripte sowie die begleitenden Dokumente werden versioniert über das folgende GitHub-Repository bereitgestellt:

<https://github.com/SteffenOe/medicaltracker.git>

1.2 Aufbau der Arbeit

Die vorliegende Arbeit gliedert sich entlang des phasenorientierten Entwicklungsprozesses: Kapitel 1 (Projektdokumentation) definiert Problemstellung, Forschungsfrage, Systemscope, Technologieauswahl, Risikomanagement und die gewählte Entwicklungsmethodik. Anschließend analysiert Kapitel 2 (Anforderungsdokumentation) die Stakeholder, spezifiziert funktionale User Stories inklusive Akzeptanzkriterien, modelliert das Anwendungsfalldiagramm und legt Qualitätsanforderungen nach

ISO/IEC 25010 fest. In Kapitel 3 (Spezifikationsdokumentation) werden das Datenmodell, die Geschäftsprozesse (UML-Aktivitätsdiagramm), die Geschäftsregeln sowie die Schnittstellen definiert. Kapitel 4 führt Limitationen, Datenschutzimplikationen und Handlungsempfehlungen zusammen und schließt mit einer Beantwortung der Forschungsfrage im Fazit ab.

1.3 Abgrenzung des Systembereichs

Zur Vermeidung von Fehlinterpretationen bezüglich des klinischen Einsatzes und des rechtlichen Rahmenwerks wird der Systemumfang nachfolgend explizit abgegrenzt:

1. Positiv-Abgrenzung (In-Scope):

- Anwendungsbereich: Rein privater Einsatz im häuslichen Pflegeumfeld zur Dokumentations- und Orientierungsunterstützung.
- Funktionsumfang: Erfassung und Bearbeitung von Medikationsdaten, Aufzeichnung von Vitalwerten, Generierung strukturierter PDF-Exportberichte für den Arztbesuch sowie ein vereinfachter Patientenmodus.
- Betriebsmodus: Standalone-Betrieb auf einem lokalen Computer innerhalb eines einzelnen Haushalts.

2. Negativ-Abgrenzung (Out-of-Scope):

- Keine medizinische Diagnostik und kein Therapiesystem: Das System stellt explizit kein Medizinprodukt dar. Es führt keine automatischen medizinischen Diagnosen, Dosisberechnungen oder Therapieempfehlungen durch. Die klinische Verantwortung verbleibt ausnahmslos beim behandelnden Fachpersonal.
- Keine Nutzung in professionellen Pflegediensten: Das System ist nicht für den kommerziellen Schicht- oder Dienstbetrieb ambulanter oder stationärer Pflegeeinrichtungen ausgelegt (keine Mandantenfähigkeit, keine Abrechnungsschnittstellen).
- Kein Mehrnutzer- / Multi-Tenancy-Betrieb: Die Anwendung verzichtet im aktuellen MVP-Status auf eine Benutzerverwaltung mit komplexer Rechte- und Rollenmatrix auf Datenbankebene.
- Keine Cloud-Infrastruktur & Remote-Zugriff: Es erfolgt keine Datenübertragung an externe Server und Cloud-Dienste.

1.4 Entwicklungsmethodik

Um eine hohe Softwarequalität und Termintreue im Rahmen des Projekts zu gewährleisten, basiert die Entwicklung auf einem strukturierten Methodenset. Nachfolgend werden das angewendete Vorgehensmodell sowie die Methodik zur Anforderungs- und Spezifikationsanalyse systematisch zusammengeführt.

1.4.1 Vorgehensmodell

Die Abwicklung des Entwicklungsvorhabens erfolgt entlang eines phasenorientierten, sequenziellen Vorgehensmodells in Anlehnung an das klassische Wasserfall- bzw. Stufenmodell. Die Wahl dieses linearen Ansatzes begründet sich durch die festen Meilensteine sowie die Notwendigkeit, vor Beginn der Implementierungsphase ein vollständig spezifiziertes und architektonisch abgesichertes Anforderungsdokument vorliegen zu haben.

Das Vorgehensmodell gliedert sich zeitlich wie inhaltlich in drei aufeinander aufbauende Sequenzen. Die erste Sequenz umfasst die Konzeption und Spezifikation in den Wochen 1 bis 2 (Phase 1). In diesem Zeitraum werden eine systematische Problem- und Stakeholder-Analyse durchgeführt, die funktionalen sowie nichtfunktionalen Anforderungen erhoben, UML-Diagramme modelliert sowie Geschäftsregeln und Schnittstellen präzise definiert. Darauf aufbauend erstreckt sich über die Wochen 3 bis 6 die zweite Sequenz der Softwarearchitektur und Implementierung (Phase 2). In dieser Phase wird die Systemarchitektur mittels UML-Sequenzdiagrammen entworfen und das Repository aufgesetzt. Darauf folgen die Backend-Entwicklung mit Flask und SQLAlchemy, die Anbindung der SQLite-Datenbank sowie die Gestaltung des Frontends unter Nutzung von Jinja2 und Bootstrap. Die dritte Sequenz umfasst in den Wochen 7 bis 8 (Phase 3) die Verifikation, das Testing und die Finalisierung. Neben Modul- und Systemtests umfasst dieser Abschnitt die Projektdokumentation samt Benutzeranleitung und Erfassung technischer Schulden. Zudem werden das Projektabstract verfasst und die Artefakte final konsolidiert.

1.4.2 Methodik

Zur systematischen Erarbeitung der Artefakte werden etablierte Standardmethoden des Software Engineerings miteinander kombiniert. Die Erhebung der funktionalen Anforderungen erfolgt aus Sicht der identifizierten Akteure in Form agiler User Stories, die gemäß dem IREB-Standard strukturiert werden. Um eine fokussierte Softwareentwicklung zu garantieren und Terminrisiken zu minimieren, werden alle Anforderungen im Hinblick auf das Minimum Viable Product (MVP) in die Prioritätsstufen Hoch, Mittel und Niedrig eingestuft. Als Anforderungen hoher Priorität gelten dabei die unverzichtbaren Kernfunktionen für den primären Systemzweck. Hierunter fallen die Medikationsverwaltung, die Vitaldatenerfassung, eine barrierefreie Tagesansicht sowie grundlegende

Sicherheitsfunktionen (FA-01, FA-02, FA-05, FA-06, FA-07 sowie NFA-01 bis NFA-07). Anforderungen mittlerer Priorität umfassen wichtige Zusatzfunktionen für den erweiterten Praxiseinsatz, wie etwa die Generierung eines PDF-Arztberichts oder die Terminverwaltung (FA-03, FA-04). Anforderungen mit niedriger Priorität werden für den Rahmen des aktuellen Entwicklungsvorhabens zurückgestellt und als Out-of-Scope eingeordnet.

Die visuelle System- und Prozessmodellierung erfolgt mittels Anwendungsfall-, Klassen- und Aktivitätsdiagrammen mit Swimlanes. Das Qualitäts- und Risikomanagement stützt sich hinsichtlich der Qualitätskriterien auf das internationale Standardmodell ISO/IEC 25010:2023. Projektrisiken werden quantitativ über eine zweidimensionale Risikomatrix aus dem Produkt von Eintrittswahrscheinlichkeit und Auswirkung evaluiert und mit entsprechenden Präventivmaßnahmen versehen.

1.5 Technologie & Systemarchitektur

Zur Gewährleistung einer hohen Erfüllung der funktionalen und nichtfunktionalen Anforderungen sowie zur Vermeidung technischer Risiken wird für die Anwendung eine klassische Client-Server-Architektur auf Basis eines reinen Python-Ökosystems gewählt:

Backend

Programmiersprache Python (v3.14+) in Kombination mit dem leichtgewichtigen Web-Framework Flask. Flask ermöglicht eine saubere objektorientierte Strukturierung der Anwendungslogik und bietet maximale Kontrolle über Routing und Datenverarbeitung.

Frontend

Die Benutzeroberfläche wird mittels der Template-Engine Jinja2 serverseitig gerendert. Das UI-Styling basiert auf Bootstrap 5, wodurch ein modernes, responsives und barrierefreies Layout ohne den Overhead externer JavaScript-Frameworks realisiert werden kann.

Datenhaltung

Als Datenbanksystem kommt SQLite zum Einsatz. SQLite arbeitet dateibasiert, erfordert keine Serverinstallation beim Endanwender und ermöglicht eine datenschutzkonforme, lokale Datenspeicherung. Die Anbindung an das Python-Backend erfolgt isoliert über den Objekt-Relationalen Mapper (ORM) Flask-SQLAlchemy.

1.6 Risikomanagement

Zur frühzeitigen Identifikation und Reduktion von Projektrisiken wird eine systematische Risikoanalyse durchgeführt. Die Risiken werden nach Eintrittswahrscheinlichkeit und Auswirkung bewertet und mit konkreten Gegenmaßnahmen (Mitigation) versehen.

Tabelle 1 Risikoanalyse

Risiko-ID	Beschreibung	Eintrittswahrscheinlichkeit	Auswirkung	Gegenmaßnahme
R-01 (Technisch)	Datenverlust / - korrup-tion Plötzlicher Serverabsturz beschädigt die dateibasierte SQLite-Datenbank.	Gering	Hoch	Einsatz von SQLAlchemy für sicheres Transaktionsmanagement; automatisierte Erstellung eines Backups der .db-Datei bei jedem App-Start.
R-02 (Design)	Mangelnde Usability (Barrierefreiheit) Die ältere Zielgruppe (Patienten) scheitert an zu kleinen Klickelementen oder komplexer Menüführung.	Mittel	Hoch	Konsequenter „Mobile-First“- und High-Contrast-Ansatz mit Bootstrap 5; Erstellung von UI-Prototypen vor der finalen Programmierung.
R-03 (Ressourcen)	Verzögerungen durch externe Faktoren Berufliche Spitzenbelastungen blockieren die Weiterentwicklung innerhalb des Zeitplans.	Mittel	Mittel	Definition eines Minimal Viable Products (MVP). Kernfunktionen (Medikation & Vitalwerte) besitzen höchste Priorität; optionale Features (PDF-Export) dienen als Puffer.

Quelle: eigene Darstellung

Die Auswertung der Risikomatrix zeigt, dass insbesondere das Design-Risiko (R-02) eine hohe Auswirkung auf den Gesamterfolg und den Kundennutzen der Anwendung hat. Durch die frühzeitige Berücksichtigung von Richtlinien zur Barrierefreiheit wird diesem Risiko bereits im Entwurf gezielt entgegengewirkt.

1.7 Zeitplanung

Die zeitliche Strukturierung des Projekts orientiert sich an einer veranschlagten Netto-Projektlaufzeit von 8 Wochen. Der Zeitplan verteilt die wesentlichen Arbeitspakete auf die drei vorgegebenen Phasen des Portfoliokurses.

Tabelle 2 stellt die Zeitplanung, die zugehörigen Arbeitspakete sowie die zentralen Qualitäts-Meilensteine (Einreichungen) dar.

Tabelle 2 Zeitplanung

Phase	Zeitraum	Wesentliche Arbeitspakete & Meilensteine
Phase 1: Konzeption	Woche 1-2	• Erstellung der Projektdokumentation (Einleitung, Risikomanagement, Zeitplan) • Erstellung des Anforderungsdokuments (Stakeholder, User Stories, NFAs, Glossar) • Erstellung des Spezifikationsdokuments (Datenmodell, Geschäftsprozesse, Schnittstellen) • Modeling der UML-Diagramme (Use-Case, Klasse, Aktivität) • Meilenstein 1: Abgabe Portfolioteil 1 & Feedback-Schleife
Phase 2: Architektur & Implementierung	Woche 3-6	• Erstellung des Architekturdokuments (Technologieentscheidung, UML-Sequenzdiagramm) • Aufsetzen des GitHub-Repositories und der Ordnerstruktur • Backend-Implementierung (Flask, SQLAlchemy-Modelle, Routing) • Frontend-Implementierung (Jinja2-Templates, Bootstrap-Layouts) • Befüllen der SQLite-Datenbank mit Testdaten • Meilenstein 2: Abgabe Portfolioteil 2 & Feedback-Schleife
Phase 3: Finalisierung	Woche 7-8	• Erstellung des Testdokuments (Teststrategie, Testprotokolle) • Schreiben der Benutzeranleitung & Erfassen technischer Schulden

		• Verfassen des 2-seitigen Abstracts („Making-of“ & Reflexion) • Zusammenfügen der Dokumente & ZIP-Export für GitHub • Meilenstein 3: Finale Einreichung der Prüfungsleistung
--	--	---

Quelle: eigene Darstellung

2 Anforderungsdokumentation

Das folgende Kapitel erfasst die fachlichen und qualitativen Anforderungen an das System. Neben der Analyse der beteiligten Stakeholder werden die funktionalen Anforderungen als User Stories formuliert sowie die nichtfunktionalen Qualitätskriterien nach ISO/IEC 25010 festgelegt.

2.1 Stakeholder-Analyse

Die Stakeholder-Analyse identifiziert alle Personen, Gruppen und Organisationen, die Einfluss auf das System haben oder von dessen Nutzung direkt oder indirekt betroffen sind. In Tabelle 3 werden die drei wesentlichen Stakeholder-Gruppen hinsichtlich ihrer Systemrolle, ihrer Erwartungshaltung sowie ihrer technischen Vorkenntnisse gegenübergestellt.

Tabelle 3 Stakeholder-Analyse

Stakeholder-Gruppe	Rolle im System	Bedürfnisse & Erwartungen	Technische Vorkenntnisse
Pflegende Angehörige	Hauptnutzende	• Schnelle Erfassung von Medikationsplänen und Vitaldaten • Terminerinnerungen und lückenlose Historie • Automatisierte Erstellung von PDF-Berichten für den Arzt	Grundlegendes IT-Verständnis (Webbrowser / Smartphone)
Zu pflegende Personen	Endnutzende (Patientenansicht)	• Maximale Übersichtlichkeit und Barrierefreiheit • Große Schriften, hohe Kontraste und einfache Interaktion • Klare Fokus-Frage: „Welche Medikamente muss ich jetzt einnehmen?“	Geringe bis sehr geringe IT-Affinität

Ärztliches Fachpersonal	Passiv (empfangend)	• Übersichtliche Zusammenfassung von Vitalwerten (z. B. Blutdruckverläufe) • Nachvollziehbarkeit der Einnahmetreue (Compliance)	Keine direkte Systemnutzung (erhält ausgedrucktes/digitales PDF)
-------------------------	---------------------	--	--

Quelle: eigene Darstellung

Aus dieser Gegenüberstellung wird deutlich, dass das System zwei stark differierende Nutzungskontexte bedienen muss: Während für pflegende Angehörige der Funktionsumfang und die effiziente Datenverwaltung im Vordergrund stehen, erfordert die Interaktion mit den zu pflegenden Personen eine strikte Reduktion der kognitiven Belastung.

2.2 Funktionale Anforderungen

Die funktionalen Anforderungen beschreiben das Soll-Verhalten der Anwendung aus Perspektive der jeweiligen Anwenderrollen. Zur lückenlosen Spezifikation der funktionalen Anforderungen dienen User Stories nach dem etablierten IREB-Standard (Pohl & Rupp, 2021). Diese werden im Hinblick auf das Minimum Viable Product (MVP) nach fachlicher Relevanz priorisiert.

FA-01: Medikationsverwaltung (Priorität: Hoch)

User Story: „Als pflegende Person möchte ich Medikamente mit Name, Dosierung, Einnahmezeitpunkt und Bestand anlegen, bearbeiten und löschen, um den Medikationsplan stets aktuell zu halten.“

Fachlicher Kontext: Unvollständige oder veraltete Pläne führen im Pflegealltag zu Medikationsfehlern. Das System stellt strukturierte Eingabemasken bereit, die Pflichtfelder erzwingen.

Akzeptanzkriterien: Die Absendung des Formulars gelingt nur, wenn alle Pflichtfelder vollständig und im korrekten Datentyp ausgefüllt sind. Nach dem Speichern erscheint der neue Datensatz unverzüglich in der Medikationsübersicht. Änderungen oder Löschungen werden ohne Datenverlust synchron in der SQLite-Datenbank reflektiert.

FA-02: Vitaldaten-Tracking (Priorität: Hoch)

User Story: „Als pflegende Person möchte ich tägliche Vitalwerte (Blutdruck, Puls, Gewicht) mit Zeitstempel erfassen und in Tabellen einsehen, um gesundheitliche Trends frühzeitig zu erkennen.“

Fachlicher Kontext: Das Erfassen strukturierter Messwerte liefert dem behandelnden Fachpersonal essenzielle Anhaltspunkte für klinische Bewertungen.

Akzeptanzkriterien: Das System versieht jeden abgespeicherten Messwert automatisch mit dem aktuellen Server-Zeitstempel. Texteingaben in numerischen Feldern werden serverseitig abgefangen. Die erfassten Messwerte werden in der historischen Verlaufstabelle chronologisch absteigend sortiert dargestellt.

FA-03: Arzt-Bericht / PDF-Export (Priorität: Mittel)

User Story: „Als pflegende Person möchte ich gefilterte Vital- und Medikationsdaten für einen wählbaren Zeitraum als PDF exportieren, um diese beim Hausarztbesuch vorzulegen.“

Fachlicher Kontext: Arztbesuche leiden häufig unter Zeitmangel und unvollständigen Patientenauskünften. Der automatisierte Export aggregiert alle relevanten Historien auf Knopfdruck.

Akzeptanzkriterien: Die anwendende Person kann das Start- und Enddatum des Berichtszeitraums über einen Datumsfilter einschränken. Das generierte PDF-Dokument enthält Stammdaten, verordnete Medikamente sowie tabellarische Vitalwertverläufe in einem druckoptimierten Layout. Die Generierung und Bereitstellung des Downloads erfolgt innerhalb von unter zwei Sekunden.

FA-04: Termin- und Aufgabenverwaltung (Priorität: Mittel)

User Story: „Als pflegende Person möchte ich anstehende Pflegetermine sowie Aufgaben eintragen, um den Pflegealltag strukturiert zu organisieren.“

Fachlicher Kontext: Die Koordination von Arztbesuchen, Physiotherapie und pflegerischen Verpflichtungen erfordert eine verlässliche Zeitplanung.

Akzeptanzkriterien: Anstehende Termine werden chronologisch sortiert auf einem Aufgaben-Board dargestellt. Durch Anklicken einer Status-Checkbox lässt sich der Termin als erledigt kennzeichnen, woraufhin die visuelle Darstellung angepasst wird.

FA-05: Barrierefreie Tagesansicht / Patientenmodus (Priorität: Hoch)

User Story: „Als pflegende Person möchte ich eine vereinfachte Tagesansicht aufrufen können, die mir in großer Schrift nur die heute anstehenden Medikamente anzeigt, um Orientierung im Alltag zu erhalten.“

Fachlicher Kontext: Zur Förderung der Handlungsautonomie verzichtet diese Ansicht auf verschachtelte Menüstrukturen und administrative Bedienelemente.

Akzeptanzkriterien: Die Oberfläche enthält keine komplexen Navigationsleisten. Die Basis-Schriftgröße beträgt durchgängig mindestens 18 pt.

FA-06: Einnahmebestätigung (Priorität: Hoch)

User Story: „Als anwendende Person (pflegende oder zu pflegende Person) möchte ich eine anstehende Tagesdosis per Klick auf eine großflächige Schaltfläche als eingenommen markieren können, um den Vollzug der Medikation eindeutig zu dokumentieren.“

Fachlicher Kontext: Die explizite Entkopplung von der Medikationsverwaltung stellt sicher, dass das bloße Erfassen eines Medikaments vom tatsächlichen Einnahmeakt im Alltag getrennt bleibt.

Akzeptanzkriterien: Der Klick auf die Bestätigungsschaltfläche löst eine Statusänderung der spezifischen Einnahme für den aktuellen Tag aus. Das UI kennzeichnet die Kachel sofort als eingenommen inklusive des genauen Ausführungszeitstempels. Die Einnahme wird unveränderlich im Protokoll verankert.

FA-07: Automatische Bestandsaktualisierung & Schwellenwertprüfung (Priorität: Hoch)

User Story: „Als pflegende Person möchte ich, dass der Systembestand eines Medikaments bei jeder Einnahmebestätigung automatisch dekrementiert wird und bei Unterschreiten des Mindestbestands eine Warnmeldung erscheint, um Versorgungsengpässe zu vermeiden.“

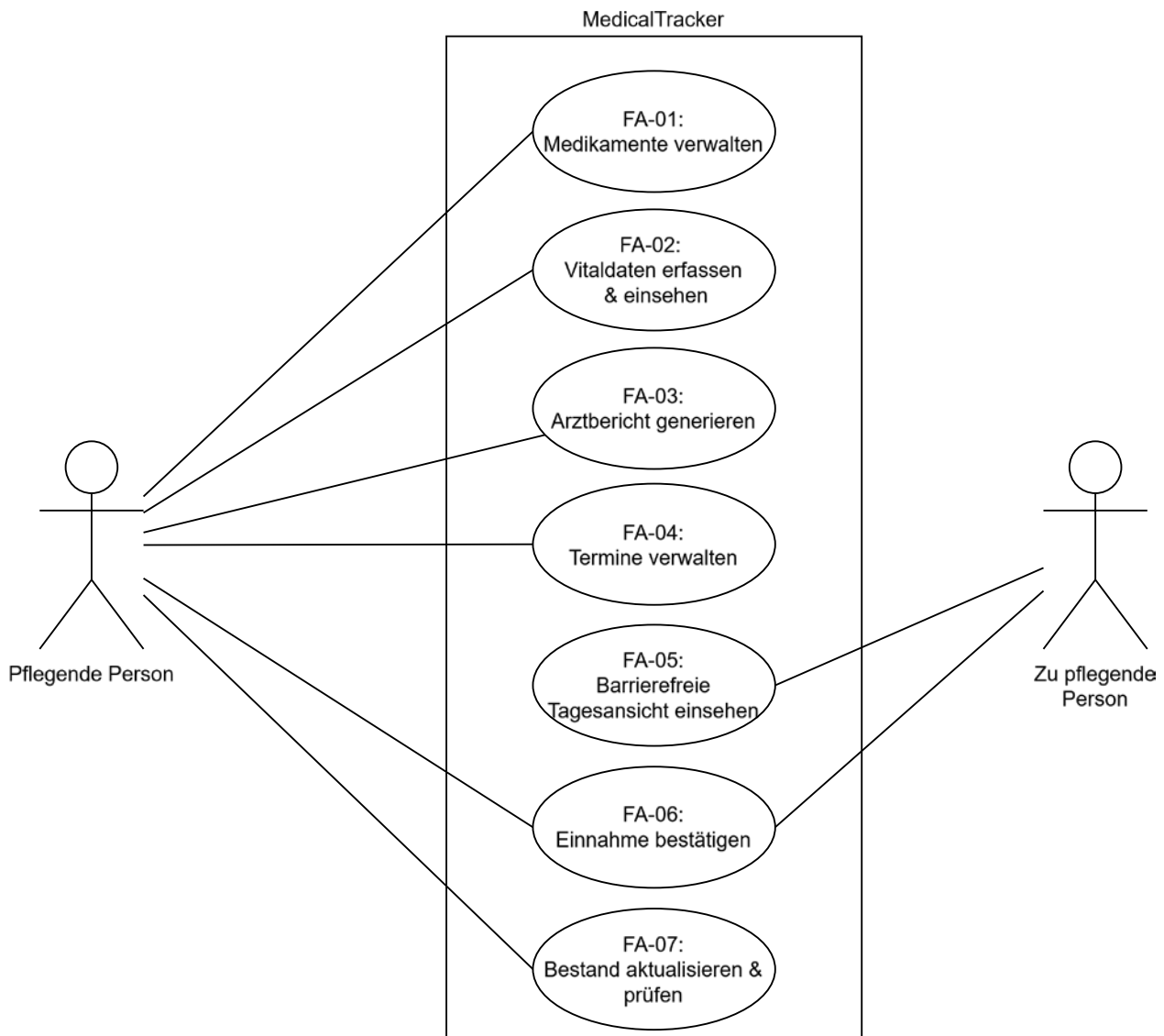
Fachlicher Kontext: Die automatisierte Koppelung von Einnahmeakt und Lagerbestand verhindert das unbemerkte Auslaufen wichtiger Arzneimittel.

Akzeptanzkriterien: Bei jeder erfolgreichen Ausführung von FA-06 reduziert das Backend den Wert des Attributs *bestand* um die verabreichte Einheitsmenge. Ergibt die anschließende Prüfung, dass $bestand \leq mindestbestand$ gilt, wird auf dem Dashboard der pflegenden Person unverzüglich ein optisch hervorgehobener Nachbestell-Warnhinweis generiert.

2.3 UML-Anwendungsfalldiagramm

Zur visuellen Abgrenzung der Systemgrenzen sowie zur Verdeutlichung der Interaktionen zwischen den identifizierten Akteuren und den funktionalen Anforderungen dient das in Abbildung 1 dargestellte UML-Anwendungsfalldiagramm.

Abbildung 1 Anwendungsfalldiagramm



Quelle: eigene Darstellung

Aus der Modellierung geht die funktionale Rollentrennung hervor: Während die pflegende Person als Hauptakteur Vollzugriff auf alle administrativen Verwaltungs- und Analysefunktionen (FA-01 bis FA-04, FA-06 und FA-07) besitzt, beschränkt sich die Interaktion der zu pflegenden Person gezielt auf den barrierefreien Patientenmodus (FA-05), sowie die Bestätigung der Einnahme (FA-06). Dies unterstützt das Ziel der kognitiven Entlastung der zu pflegenden Person.

2.4 Nichtfunktionale Anforderungen

Nichtfunktionale Anforderungen definieren die Qualitätskriterien, Rahmenbedingungen und Performanz-Grenzen, die das System einhalten muss. Zur wissenschaftlichen Fundierung orientiert sich die Klassifikation an der Norm ISO/IEC 25010 (ISO/IEC, 2023). Tabelle 4 führt die nichtfunktionalen Anforderungen auf.

Tabelle 4 Nichtfunktionale Anforderungen

NFA-ID	Kategorie	Anforderung & Spezifikation
NFA-01	Nutzbarkeit (Interaction Capability)	Die Gestaltung des Patientenmodus orientiert sich an den Richtlinien der Barrierefreiheit der WCAG 2.2 (W3C, 2024). Es wird ein hohes Kontrastverhältnis eingehalten. Für den Patientenmodus werden großflächige Interaktionselemente sowie eine projektspezifische Basis-Schriftgröße von mindestens 18 pt vorgegeben.
NFA-02	Leistungsfähigkeit (Performance Efficiency)	Die Antwortzeit des lokalen Flask-Servers darf beim Laden von Ansichten unter Windows 10/11 maximal 1 Sekunde betragen. Lese- und Schreibzugriffe auf die SQLite-Datenbank müssen unter 200 ms liegen.
NFA-03	Sicherheit (Security)	Alle personenbezogenen Gesundheitsdaten verbleiben ausschließlich in der lokalen SQLite-Datenbank (Privacy by Design). Formulardaten werden gegen SQL-Injection (via SQLAlchemy ORM) und XSS (via Jinja2 Auto-Escaping) geschützt.
NFA-04	Zuverlässigkeit (Reliability)	Das System darf bei fehlerhaften Benutzereingaben (z. B. Buchstaben im Blutdruckfeld) nicht abstürzen. Fehleingaben werden abgefangen und durch visuelle Hinweise im Formular quittiert.
NFA-05	Flexibilität (Flexibility)	Die Anwendung muss ohne externe Serverinfrastruktur nativ im Standard-Webbrowser unter Windows 10/11 lauffähig sein. Ein lokaler Python-Interpreter (v3.14+) ist als Ausführungsumgebung ausreichend.
NFA-06	Wartbarkeit (Maintainability)	Strikte Trennung der Zuständigkeiten nach dem Model-View-Controller-Muster (MVC) via Flask-Blueprints. Einhaltung des PEP-8-Coding-Standards mit einem Pylint-Score von $\geq 9,0/10$. Testbarkeit: Die automatisierte Testabdeckung (Unit- & Integrationstests via pytest) der Kern-Geschäftslogik (Validatoren, Bestandsberechnungen) muss mindestens 80 % (Line

		Coverage) betragen. Die Komplexität (McCabe-Metrik) darf $M \leq 10$ pro Funktion nicht überschreiten.
NFA-07	Skalierbarkeit (Scalability)	Die Datenarchitektur muss so ausgelegt sein, dass das System bis zu 10.000 historische Vitaldaten- und Protokolleinträge pro Patient ohne Performanzeinbußen verarbeiten kann. Eine vertikale Skalierung auf eine Server-Datenbank (z. B. PostgreSQL) muss ohne Änderung des Datenmodells möglich sein.

Quelle: eigene Darstellung

2.5 Datenschutz- und Sicherheitsanalyse

Die Verarbeitung von Gesundheitsdaten unterliegt aufgrund ihres sensiblen Charakters strengen rechtlichen und technischen Anforderungen. Im bisherigen Softwaredesign wurde die Aussage der Datenschutzkonformität vorrangig auf die lokale Datenhaltung zurückgeführt. Eine wissenschaftlich fundierte Betrachtung erfordert jedoch die explizite Differenzierung der rechtlichen Grundlagen nach der Datenschutz-Grundverordnung (DSGVO), die konkrete Spezifikation technischer und organisatorischer Maßnahmen (TOMs) sowie die Analyse der Hürden für einen potenziellen klinischen Realeinsatz.

2.5.1 Einordnung nach DSGVO

Gesundheitsdaten wie verordnete Medikamente, Einnahmeprotokolle und Vitalwerte stellen besondere Kategorien personenbezogener Daten im Sinne von Art. 9 Abs. 1 DSGVO dar. Die Zulässigkeit der Verarbeitung stützt sich auf zwei Säulen:

Die rechtliche Privat- und Haushaltsausnahme (Art. 2 Abs. 2 lit. c DSGVO)

Da die Anwendung ausschließlich für den privaten Gebrauch durch pflegende Angehörige und zu pflegende Personen innerhalb eines Haushalts konzipiert ist, greift primär die sogenannte Haushaltsausnahme. Diese entbindet rein persönliche oder familiäre Tätigkeiten von den administrativen Pflichten der DSGVO (wie dem Führen eines Verzeichnisses). Die strikte Vermeidung jeglicher Datenübertragung an externe Server oder Cloud-Dienste stellt sicher, dass dieser rechtliche Schutzraum nicht verlassen wird.

Explizite Einwilligung und Datenminimierung (Art. 6 Abs. 1 lit. a & Art. 9 Abs. 2 lit. a DSGVO)

Sollten Daten im häuslichen Umfeld zwischen der pflegenden und der zu pflegenden Person ausgetauscht werden, erfolgt dies auf Basis einer freiwilligen, expliziten Einwilligung. Das System setzt das Prinzip der Datenminimierung (Privacy by Design) konsequent um: Es werden ausschließlich diejenigen Attribute verarbeitet, die für den Zweck der Pflegedokumentation zwingend erforderlich sind.

2.5.2 Technische und organisatorische Schutzmaßnahmen

Die reine Zusage einer lokalen Speicherung reicht zur Gewährleistung der Datensicherheit nicht aus, da auch lokale Systeme Bedrohungen wie unbefugtem physischen Zugriff, Schadsoftware oder Datenverlust ausgesetzt sind. Das Sicherheitskonzept spezifiziert daher konkrete Schutzmaßnahmen gemäß Art. 32 DSGVO:

Vertraulichkeit & Zugriffsschutz: Auf Anwendungsebene wird das Ausführen von SQL-Befehlen isoliert über den Objekt-Relationalen Mapper Flask-SQLAlchemy abgewickelt. Dies verhindert nicht nur SQL-Injection-Angriffe, sondern stellt sicher, dass keine unvalidierten Rohdaten in das Dateisystem gelangen. Das Betriebssystem schützt die SQLite-Datenbankdatei vor unbefugtem Auslesen. Hierzu setzt das Konzept auf lokale Zugriffsrechte (User Account Control) und empfiehlt eine Festplattenverschlüsselung. (z. B. via Windows BitLocker).

Integrität & Eingabesicherheit: Um Angriffe über die Benutzeroberfläche auszuschließen, kommen client- und serverseitige Filtermechanismen zum Einsatz. Das automatische Escaping der Jinja2-Template-Engine unterbindet Cross-Site-Scripting (XSS). Formulareingaben werden mittels CSRF-Tokens (Cross-Site Request Forgery via Flask-WTF) gegen ungewollte Anfragen geschützt.

Verfügbarkeit & Belastbarkeit: Zur Absicherung gegen Datenkorruption oder plötzliche Systemabstürze führt die Anwendung bei jedem Start eine automatisierte Integritätsprüfung des Dateikopfs der SQLite-Datenbank durch. Gleichzeitig wird vor dem Schreibzugriff eine automatisierte lokale Sicherheitskopie der Datenbankdatei in einem isolierten Backup-Ordner angelegt.

Evaluierung & Transaktionssicherheit: Fehlerhafte Transaktionen lösen einen sofortigen Rollback-Befehl aus, wodurch der Datenbankzustand bei Validierungsfehlern konsistent bleibt.

2.5.3 Anforderungen an einen potenziellen klinischen Realeinsatz

Sollte der MedicalTracker aus dem privaten Umfeld in den professionellen klinischen Betrieb (z. B. in Pflegediensten oder Arztpraxen) überführt werden, ergeben sich zusätzliche rechtliche und architektonische Anforderungen:

1. Zertifizierung als Medizinprodukt (MDR 2017/745): Das System müsste ein Konformitätsbewertungsverfahren durchlaufen, um die Anforderungen an Risikomanagement (ISO 14971) und Software-Lebenszyklusprozesse (IEC 62304) nachzuweisen.
2. Kryptographische Verschlüsselung (At-Rest & In-Transit): Die SQLite-Datenbank müsste auf Dateiebene verschlüsselt werden. Für Schnittstellen zu Praxissystemen wäre eine Ende-zu-Ende-Verschlüsselung nach TLS 1.3 zwingend erforderlich.
3. Rollennavigations- & Audit-Trail-System: Es müsste eine differenzierte Rechte- und Rollenmatrix (RBAC) mit strikter Trennung zwischen Pflegepersonal, ärztlichem Fachpersonal und zu pflegenden Personen implementiert werden. Sämtliche Lese- und Schreibzugriffe müssten in einem revisionssicheren Logbuch (Audit Trail) protokolliert werden.
4. Standardisierte Gesundheitsschnittstellen: Der Datenaustausch müsste über internationale Standards der Medizininformatik wie HL7 FHIR (Fast Healthcare Interoperability Resources) abgewickelt werden, um eine Interoperabilität mit der elektronischen Patientenakte (ePA) herzustellen.
5. Datenschutz-Folgenabschätzung (DSFA nach Art. 35 DSGVO): Vor Inbetriebnahme im klinischen Kontext wäre eine systematische Risikofolgenabschätzung unter Einbeziehung von Datenschutzbeauftragten gesetzlich vorgeschrieben.

3 Spezifikationsdokumentation

Dieses Kapitel beschreibt das objektorientierte Datenmodell der zentralen Geschäftsobjekte sowie deren fachliche Beziehungen und Kardinalitäten. Des Weiteren werden die dynamischen Geschäftsprozesse – mit besonderem Fokus auf den Kernprozess der Medikamenteneinnahme und Bestandsführung – detailliert dargelegt. Ergänzend werden die strikt einzuhaltenden Geschäftsregeln formuliert sowie die technischen System- und Benutzerschnittstellen spezifiziert.

3.1 Objektorientiertes Datenmodell & Geschäftsobjekte

Das Datenmodell bildet das statische Fundament der Anwendung und strukturiert die zentralen Geschäftsobjekte sowie deren relationale Verknüpfungen. Zur Behebung struktur- und ablauftechnischer Unschärfen wird die Entität der Einnahmeprotokollierung explizit im Schema verankert und in ihrer relationalen Beziehung zu den Stammdaten der Medikation und des Patienten präzisiert.

1. Geschäftsobjekt: Patient

Fungiert als primärer Ankerpunkt des relationalen Modells. Über den Primärschlüssel *id* stehen alle personenbezogenen Gesundheitsdaten in einer eindeutigen 1:n-Beziehung zum Patienten.

- *id* (Integer, Primary Key): Eindeutiger Identifikationsschlüssel.
- *vorname* (String): Vorname der zu pflegenden Person.
- *nachname* (String): Nachname der zu pflegenden Person.
- *geburtsdatum* (Date): Geburtsdatum.
- *notfallkontakt* (String): Telefonnummer oder Name einer Kontaktperson für Notfälle.

2. Geschäftsobjekt: Medikament

Repräsentiert die verordneten Arzneimittel und verweist über den Fremdschlüssel *patient_id* auf den zugehörigen Patienten. Während die Entität Medikament den aktuellen Soll-Zustand und den verfügbaren Gesamtvorrat beschreibt, erfordert die lückenlose Nachvollziehbarkeit des Ist-Zustands eine dedizierte Protokollentität.

- *id* (Integer, Primary Key): Eindeutiger Identifikationsschlüssel.
- *patient_id* (Integer, Foreign Key): Referenz auf den zugehörigen Patienten.
- *name* (String): Handelsname des Medikaments (z. B. "Ramipril 5mg").
- *dosierung* (String): Verordnete Einnahmemenge (z. B. "1 Tablette").
- *tageszeit* (String): Vorgesehener Einnahmezeitpunkt (z. B. "Morgens").
- *bestand* (Integer): Aktuell vorhandener Vorrat in Einheiten.
- *mindestbestand* (Integer): Schwellenwert für automatische Nachbestell-Warnungen.

3. Geschäftsobjekt: EinnahmeProtokoll

Verknüpft das statische Medikamentenobjekt mit dynamischen Ausführungsereignissen im Pflegealltag. Jedes Protokollobjekt besitzt einen eigenen Primärschlüssel *id* sowie die zwei Fremdschlüssel *patient_id* und *medikament_id*. Durch diese Modellierung bleibt die historische Dokumentation auch dann unverändert erhalten, wenn Stammdaten des Ursprungsmedikaments angepasst oder gelöscht werden.

- id (Integer, Primary Key): Eindeutiger Identifikationsschlüssel
- patient_id (Integer, Foreign Key): Referenz auf den zugehörigen Patienten.
- medikament_id (Integer, Foreign Key): Referenz auf das Medikament.
- soll_tageszeit (String): Geplante Einnahmezeit
- einnahme_zeitpunkt (DateTime): Tatsächliche Einnahmezeit
- erledigt (Boolean): Boolesche Flag, ob die Einnahme erfolgt ist.

4. Geschäftsobjekt: Vitalwert

Erfasst eine einzelne medizinische Messung zu einem konkreten Zeitpunkt und verweist über den Fremdschlüssel *patient_id* auf den zentralen Patienten.

- id (Integer, Primary Key): Eindeutiger Identifikationsschlüssel.
- patient_id (Integer, Foreign Key): Referenz auf den zugehörigen Patienten.
- zeitpunkt (DateTime): Datum und Uhrzeit der Messung.
- kategorie (String): Art der Messung (z. B. "Blutdruck Systolisch", "Puls", "Gewicht").
- wert (Float): Gemessener numerischer Wert.
- einheit (String): Maßeinheit des Werts (z. B. "mmHg", "bpm", "kg").

5. Geschäftsobjekt: Termin

Speichert geplante Pflegetermine und Aufgaben im Alltagsplaner und verweist ebenfalls über den Fremdschlüssel *patient_id* auf den zentralen Patienten.

- id (Integer, Primary Key): Eindeutiger Identifikationsschlüssel.
- patient_id (Integer, Foreign Key): Referenz auf den zugehörigen Patienten.
- titel (String): Kurzbeschreibung des Termins (z. B. "Arzttermin Dr. Weber").
- zeitpunkt (DateTime): Ansetzungszeitpunkt.
- kategorie (String): Art des Termins (z. B. "Arzt", "Physiotherapie").
- erledigt (Boolean): Bearbeitungsstatus (Ja/Nein).

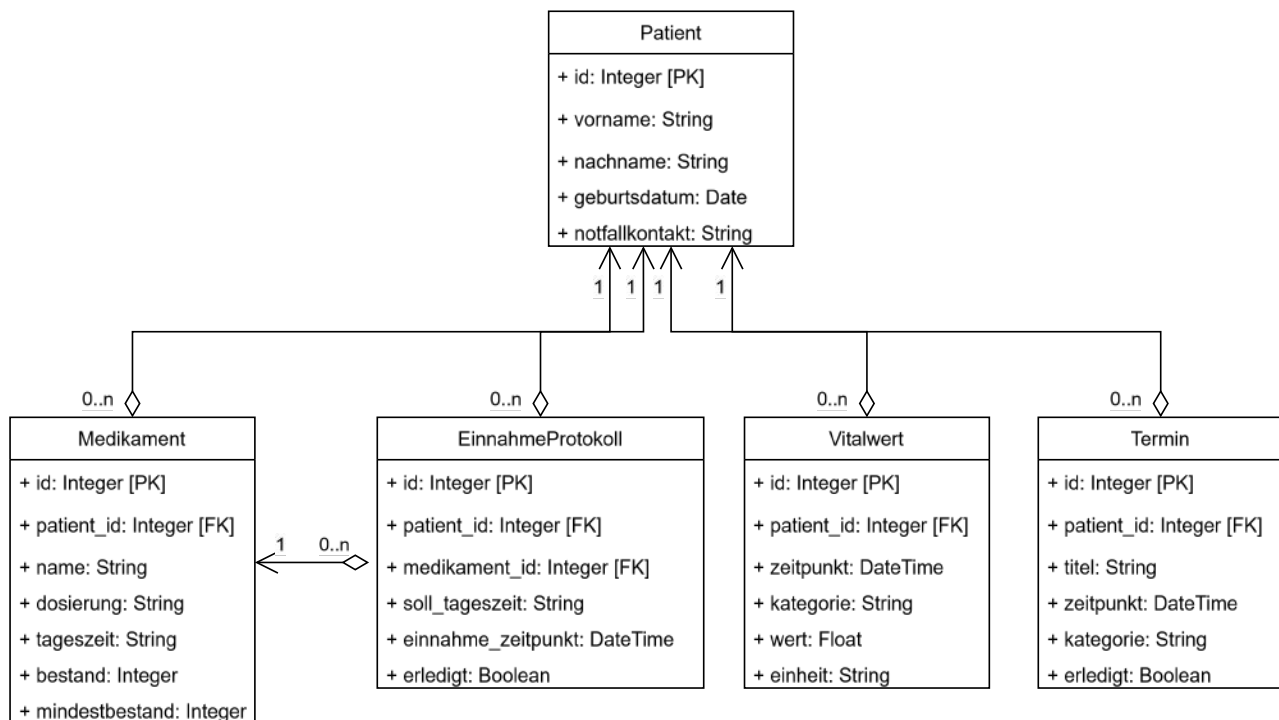
Multiplizitäten & Fachliche Beziehungen

Zwischen den Geschäftsobjekten bestehen klare 1:n-Kardinalitäten:

- Ein Patient besitzt beliebig viele zugeordnete Medikamente.
- Ein Patient besitzt beliebig viele Einnahmen.
- Ein Medikament besitzt beliebig viele Einnahmen.
- Ein Patient besitzt beliebig viele historische Vitalwerte.
- Ein Patient besitzt beliebig viele geplante Termine.

Die statische Struktur des Datenmodells sowie die fachlichen Beziehungsstrukturen und Kardinalitäten zwischen den zentralen Geschäftsobjekten werden im nachfolgenden UML-Klassendiagramm (Abbildung 2) zusammenfassend abgebildet.

Abbildung 2 Klassendiagramm



Quelle: eigene Darstellung

Das Klassendiagramm verdeutlicht die zentrale Stellung des Geschäftsobjekts Patient. Die abhängigen Entitäten Medikament, Vitalwert und Termin stehen jeweils über fremdschlüsselbasierte 1:n-Kardinalitäten in direkter Relation zum Patienten. Diese Struktur garantiert, dass alle Gesundheits- und Pflegedaten im System eindeutig einer Person zugeordnet sind.

3.2 Geschäftsprozesse

Die Geschäftsprozesse beschreiben das dynamische Zusammenspiel zwischen Benutzereingaben, Systemlogik und Datenhaltung.

GP-01: Erfassung eines neuen Medikaments

Der Prozess beginnt mit dem Aufruf der Medikationsübersicht durch den pflegenden Angehörigen. Nach Klick auf die Schaltfläche „Neues Medikament“ öffnet sich ein Formular. Die anwendende Person gibt die Stammdaten (Name, Dosierung, Einnahmezeitpunkt) sowie den aktuellen Anfangsbestand und den gewünschten Mindestbestand ein. Bei der Übermittlung prüft das System die Eingaben auf Vollständigkeit und numerische Plausibilität. Nach erfolgreicher Validierung wird der Datensatz persistent in der SQLite-Datenbank gespeichert und die anwendende Person erhält eine Bestätigungsmeldung.

GP-02: Vitaldatenerfassung und Schwellenwertprüfung

Zur Dokumentation von Vitalwerten wählt die anwendende Person die entsprechende Kategorie (z. B. Blutdruck) aus. Nach der Werteingabe führt das System eine client- und serverseitige Validierung durch, um fehlerhafte Datentypen (z. B. Texteingaben) abzufangen. Ergänzend wird geprüft, ob die eingegebenen Werte innerhalb medizinisch plausibler Grenzen liegen. Liegt beispielsweise der systolische Blutdruckwert unter dem diastolischen Wert, bricht das System den Speichervorgang ab und gibt eine detaillierte Fehlermeldung aus. Validierte Werte werden zusammen mit dem aktuellen Serverzeitstempel in der Historie abgelegt.

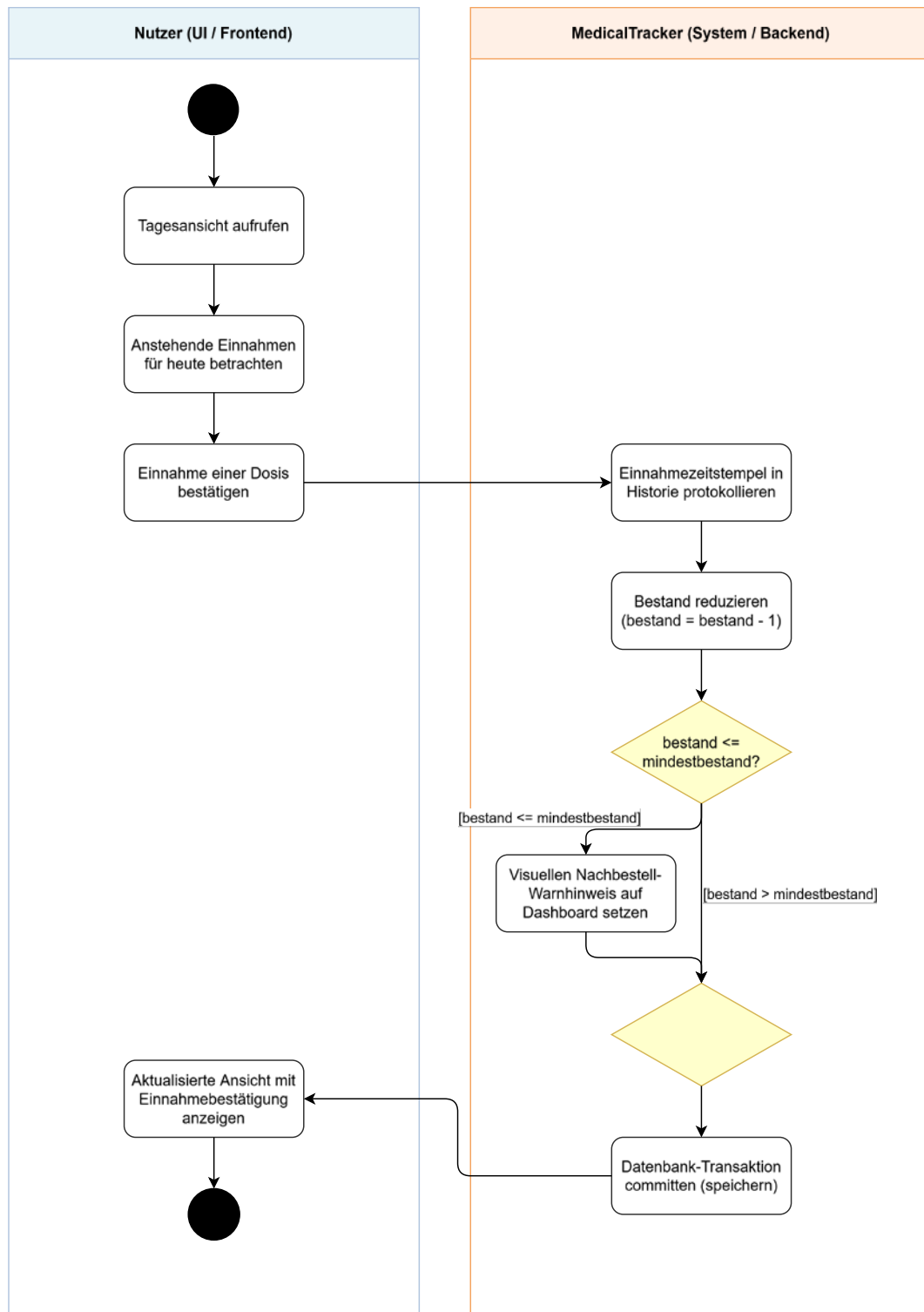
GP-03: Kernprozess - Medikamenteneinnahme und Bestandsaktualisierung

Der zentrale Geschäftsprozess verknüpft die alltägliche Pflegeinteraktion direkt mit der automatisierten Bestandsführung des Systems. Das Ablaufmuster gliedert sich in folgende Schritte:

1. **Prozessstart:** Die pflegende oder zu pflegende Person öffnet die barrierefreie Tagesansicht. Das System liest die für den aktuellen Tag terminierten Einnahmen aus der Datenbank aus und stellt diese dar.
2. **Einnahmebestätigung:** Die anwendende Person markiert eine anstehende Dosis durch Anklicken der entsprechenden Bestätigungsschaltfläche als „eingenommen“.
3. **Zustandsänderung & Transaktionsstart im Backend:** Das Backend nimmt den HTTP-POST-Request entgegen und initiiert eine atomare Datenbanktransaktion. Im selben Transaktionskontext werden zwei synchrone Operationen ausgeführt:
 - a. **Protokollerstellung:** Es wird ein neues Objekt der Entität EinnahmeProtokoll instanziiert, mit dem aktuellen Serverzeitstempel sowie den Fremdschlüsseln versehen.
 - b. **Bestandsdekrementierung:** Das zugehörige Medikament-Objekt wird geladen und dessen Attribut bestand um die verabreichte Dosis reduziert.
4. **Verzweigung & Schwellenwertprüfung:** Das System führt eine logische Verzweigung durch und prüft die Geschäftsregeln (GR-01 Bestandspositivität). Ist der Schwellenwert unterschritten, wird ein visueller Nachbestell-Warnhinweis auf dem Dashboard gesetzt.
5. **Transaktionsabschluss & Rollback:** Verläuft die Validierung fehlerfrei, wird die Transaktion dauerhaft in der Datenbank gespeichert. Schlägt ein Teilschritt fehl, führt das System ein Rollback aus. Dies verhindert eine Diskrepanz zwischen digitalem Protokoll und physischem Medikamentenbestand.

Das UML-Aktivitätsdiagramm (Abbildung 3) veranschaulicht den dynamischen Ablauf des Kernprozesses GP-03 samt Kontrollflüssen und Entscheidungslogiken. Die Unterteilung in die Swimlanes Frontend (Nutzer) und Backend (System) grenzt die Zuständigkeiten klar ab.

Abbildung 3 Aktivitätsdiagramm



Quelle: eigene Darstellung

Wie die Prozessmodellierung zeigt, koppelt das System die Interaktion im Frontend direkt mit einer automatisierten Bestandskontrolle im Backend. Durch die Auswertung der Bedingung wird sichergestellt, dass bei kritischen Beständen unverzüglich eine optische Nachbestell-Warnung auf dem Dashboard generiert wird, bevor die Datenbanktransaktion erfolgreich abgeschlossen wird.

3.3 Geschäftsregeln

Geschäftsregeln definieren fachliche Bedingungen und Invarianten, die vom System jederzeit durchgesetzt werden müssen. Prüfroutinen im Frontend, Validierungslogiken im Backend und Constraints in der SQLite-Datenbank sichern die Regeln ab. Dieses mehrstufige Validierungskonzept verhindert Fehleingaben und gewährleistet eine verlässliche Datenbasis für das Pflegepersonal. Dies umfasst Prüfroutinen auf der Benutzeroberfläche, Validierungslogiken in den Flask-Routen sowie harte Integritätsbedingungen innerhalb des SQLite-Datenbankschemas. Nachfolgend werden die zentralen Geschäftsregeln des Systems detailliert spezifiziert:

GR-01 (Bestandspositivität):

Formale Bedingung:

$$\text{bestand} \geq 0 \wedge \text{mindestbestand} \geq 0$$

Begründung:

Ein negativer Arzneimittelbestand ist physikalisch unmöglich. Negative Werte würden die automatisierte Nachbestell-Logik verfälschen und das Generieren von Warnmeldungen auf dem Dashboard blockieren oder fehlerhafte Signale auslösen.

Technische Durchsetzung:

- Clientseitig: HTML5-Eingabefelder mit dem Attribut `min="0"`.
- Serverseitig: Validierung vor Ausführung des Datenbank-Commits.
- Datenbankebene: Verankerung einer Prüfung ('bestand >= 0') im SQLAlchemy-Modell der Entität Medikament.

GR-02 (Plausibilität der Vitaldatenerfassung):

Formale Bedingung:

$$\text{Blutdruck: } \text{systolisch} > \text{diastolisch} \wedge (30 \text{ mmHg} \leq \text{diastolisch} < \text{systolisch} \leq 300 \text{ mmHg})$$

$$\text{Pulsfrequenz: } 30 \text{ bpm} \leq f_{\text{puls}} \leq 250 \text{ bpm}$$

$$\text{Körpergewicht: } 2,0 \text{ kg} \leq m \leq 300,0 \text{ kg}$$

Begründung:

Aus physiologischer Sicht beschreibt der systolische Wert den maximalen Druck in der Kontraktionsphase des Herzens, wogegen der diastolische Wert den Dauerdruck in der Entspannungsphase wiedergibt (McEvoy et al., 2024). Ein Vertauschen beider Werte würde die automatische Verlaufsanalyse für das behandelnde Fachpersonal verfälschen.

Die Schranken von 30 mmHg bis 300 mmHg stellen ein softwareseitiges Validierungsfenster dar. Es deckt das Spektrum extremer klinischer Zustände, von schweren Schockzuständen bis hin zu hypertensiven Notfällen, ab und orientiert sich an den technischen Messbereichen nach DIN EN ISO 81060-2 (Deutsches Institut für Normung, 2024) für nicht-invasive Blutdruckmessgeräte. Werte außerhalb dieses Fensters weisen mit an Sicherheit grenzender Wahrscheinlichkeit auf Fehleingaben (z. B. Tippfehler) hin. Für die Pulsfrequenz orientiert sich das Intervall an den medizinischen Extrema (von extremer Bradykardie bis hin zu paroxysmalen Tachykardien) gemäß den ESC-Leitlinien (Glikson et al., 2021) und der Gerätenorm DIN EN ISO 80601-2-61:2019. Die Grenzen für das Körpergewicht definieren ein softwareseitiges Validierungsfenster zur Vermeidung von Fehleingaben (z. B. Kommasetzungsfehler). Das Intervall deckt das gesamte Spektrum medizinischer Waagen gemäß DIN EN 45501:2015 (Eichklasse III) ab.

Technische Durchsetzung:

- Serverseitig: Eine benutzerdefinierte Validierungsfunktion im Flask-Backend prüft bei der Übermittlung die Werte.
- Fehlerbehandlung: Bei Regelverletzung wird die Transaktion abgebrochen, ein Rollback ausgeführt und das Formular mit einem spezifischen Hinweistext neu gerendert.

GR-03 (Historiensicherheit):

Formale Bedingung:

$\forall x \in \{\text{Vitalwerte}, \text{Einnahmeprotokolle}\}: \text{UPDATE}(x) = \perp \wedge \text{DELETE}(x) = \perp$

Begründung:

Einmal erfasste Vitaldaten und Einnahmebestätigungen bilden die chronologische Dokumentationsgrundlage für die behandelnde Ärzteschaft. Das nachträgliche Manipulieren oder Stornieren historischer Einträge würde die Nachvollziehbarkeit der Einnahmetreue (Compliance) gefährden und den Wert des generierten Arztberichts entkräften.

Technische Durchsetzung:

- Architektonisch: Auf API- und Routing-Ebene werden für Protokolleinträge und historische Vitalwerte keine UPDATE- oder DELETE-Endpunkte für Standard-Anwender bereitgestellt.
- Datenbankebene: Neue Messungen werden ausschließlich über INSERT-Operationen als neue Datensätze angefügt.

GR-04 (Geburtsdatum):

Formale Bedingung:

geburtsdatum < *datum*_{aktuell}

Begründung:

Das Geburtsdatum der zu pflegenden Person dient als Stammdatenbasis für die Altersberechnung und Identifikation. Ein in der Zukunft liegendes Datum ist logisch ausgeschlossen.

Technische Durchsetzung:

- Clientseitig: Einschränkung des HTML5-Datepickers durch das dynamisch gesetzte Attribut `max="YYYY-MM-DD"`.
- Serverseitig: Vergleichende Prüfung gegen `datetime.date.today()` in der Python-Geschäftslogik vor dem Speichern der Patientendaten.

3.4 Schnittstellen & Validierung

Schnittstellen bilden die definierten Übergabepunkte eines Softwaresystems zu seiner Umwelt. Sie gliedern sich in Systemschnittstellen zur technischen Kommunikation zwischen Softwarekomponenten und Benutzerschnittstellen zur Human-Computer-Interaction. Die Schnittstellen stellen sicher, dass Daten sowohl technisch sicher verarbeitet werden, als auch für zwei stark unterschiedliche Anwendergruppen ergonomisch zugänglich sind.

3.4.1 Systemschnittstellen

Die internen und externen Systemschnittstellen regeln den Datenaustausch zwischen den Komponenten der Anwendungsarchitektur. Sie stellen sicher, dass Informationen verlustfrei, typsicher und performant übertragen werden.

Internes HTTP-REST-Kommunikationsmodell (Frontend ↔ Backend)

Zweck:

Übertragung von Benutzereingaben, Formularinhalten und Navigationsbefehlen zwischen dem Browser des Anwenders und dem Flask-Backend.

Protokoll & Methoden:

Standardisiertes HTTP/1.1 via GET- und POST-Requests.

Datenformat:

Formulardaten werden als `application/x-www-form-urlencoded` an den Server übermittelt. Bei dynamischen UI-Aktualisierungen (z. B. Statusänderungen in der Tagesansicht) werden serverseitig generierte HTML-Snippets oder strukturelle JSON-Antworten (`application/json`) ausgetauscht.

PDF-Reporting-Schnittstelle (Backend ↔ Export-Engine)

Zweck:

Dynamische Erzeugung von aufbereiteten Arztberichten auf Basis historischer Datenbankeinträge.

Protokoll & Bibliotheken:

Anbindung der Python-Bibliothek ReportLab über eine interne Service-Schnittstelle.

Datenformat & Ablauf:

Die Schnittstelle nimmt aggregierte Vital- und Medikationsdaten entgegen und verarbeitet diese über eine Template-Engine. Anschließend wird das generierte PDF-Dokument (`application/pdf`) per HTTP-Header (`Content-Disposition: attachment`) als Download bereitgestellt.

Objekt-Relationale Datenbankschnittstelle (Backend ↔ SQLite)

Zweck:

Entkopplung der objektorientierten Python-Anwendungslogik vom relationalen Datenbankmodell.

Protokoll & Technologie:

Kommunikation über den Objekt-Relationalen Mapper Flask-SQLAlchemy und den nativen `sqlite3`-Treiber.

Funktionsweise:

Anfragen werden im Python-Code als objektorientierte Abfragen formuliert und vom ORM automatisch in optimierte SQL-Statements (`SELECT`, `INSERT`, `UPDATE`) übersetzt. Dies schützt das System architektonisch vor SQL-Injection-Angriffen.

3.4.2 Benutzerschnittstellen & UI-Konzept

Die Gestaltung der Benutzeroberfläche folgt einem dualen Schnittstellenkonzept, das den stark voneinander abweichenden Anforderungen der beiden primären Stakeholder-Gruppen Rechnung trägt.

Dashboard für pflegende Angehörige (Verwaltungsmodus)

Zielgruppe:

Pflegende Angehörige mit durchschnittlichen IT-Vorkenntnissen.

Struktur & Layout:

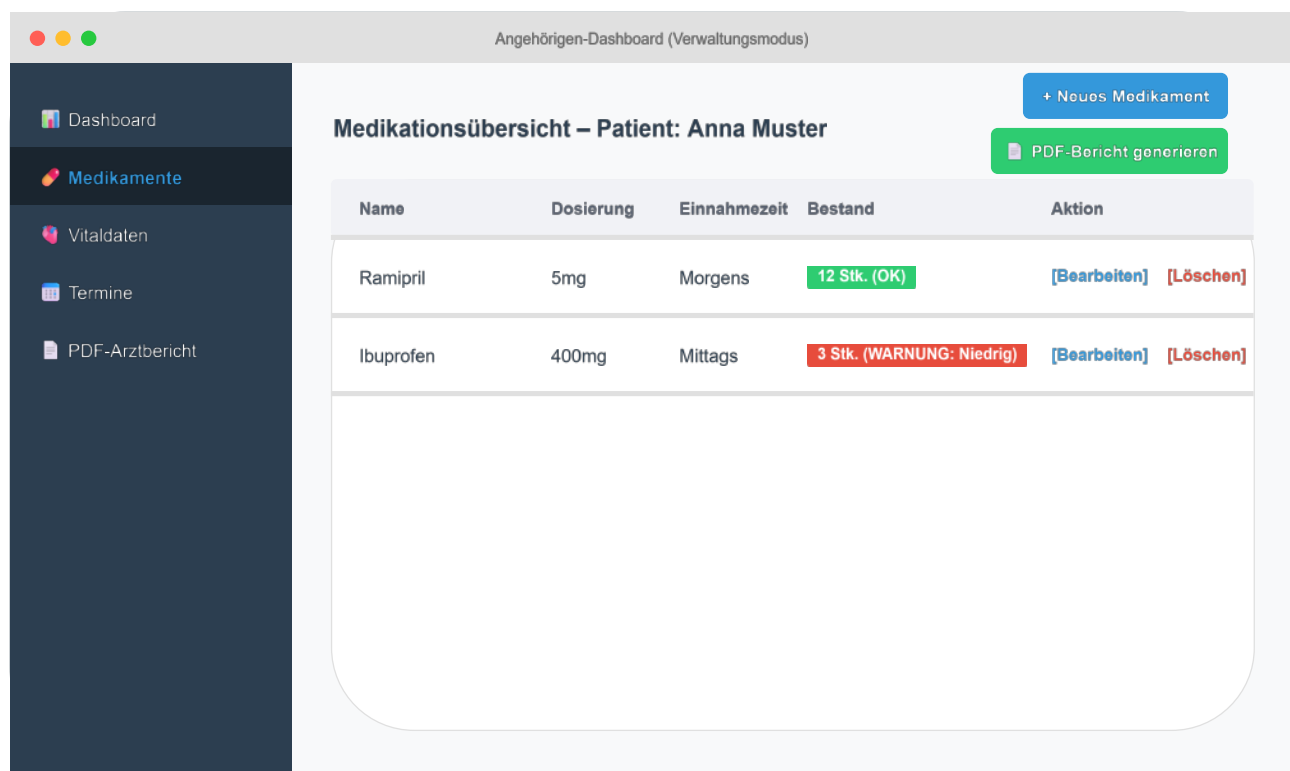
Die Oberfläche basiert auf einem responsiven Dashboard-Layout mit fixer Sidebar-Navigation. Das Design bietet eine hohe Informationsdichte und ermöglicht die effiziente Verwaltung aller Daten.

Interaktionselemente:

Tabellarische Ansichten zur Anzeige von Medikations- und Vitaldaten-Historien, Filter- und Suchfunktionen sowie modale Dialogfenster zur schnellen Erfassung neuer Datensätze, ohne die aktuelle Seite verlassen zu müssen.

Das grundlegende Verteilungsmuster und die funktionale Struktur des Verwaltungsdashboards werden in der nachfolgenden Skizze (Abbildung 4) dargestellt.

Abbildung 4 Verwaltungsdashboard



Quelle: eigene Darstellung

Das Mock-up verdeutlicht die klare Trennung von Hauptnavigation (links) und Arbeitsbereich (rechts). Durch den gezielten Einsatz von Tabellen-Aktionen und prominent platzierten Aktions-Buttons wird eine intuitive Bedienbarkeit bei administrativen Aufgaben gewährleistet.

Patientenmodus (Barrierefreie Tagesansicht)

Zielgruppe:

Zu pflegende Personen mit potenziell motorischen oder visuellen Einschränkungen und geringer IT-Affinität.

Struktur & Layout:

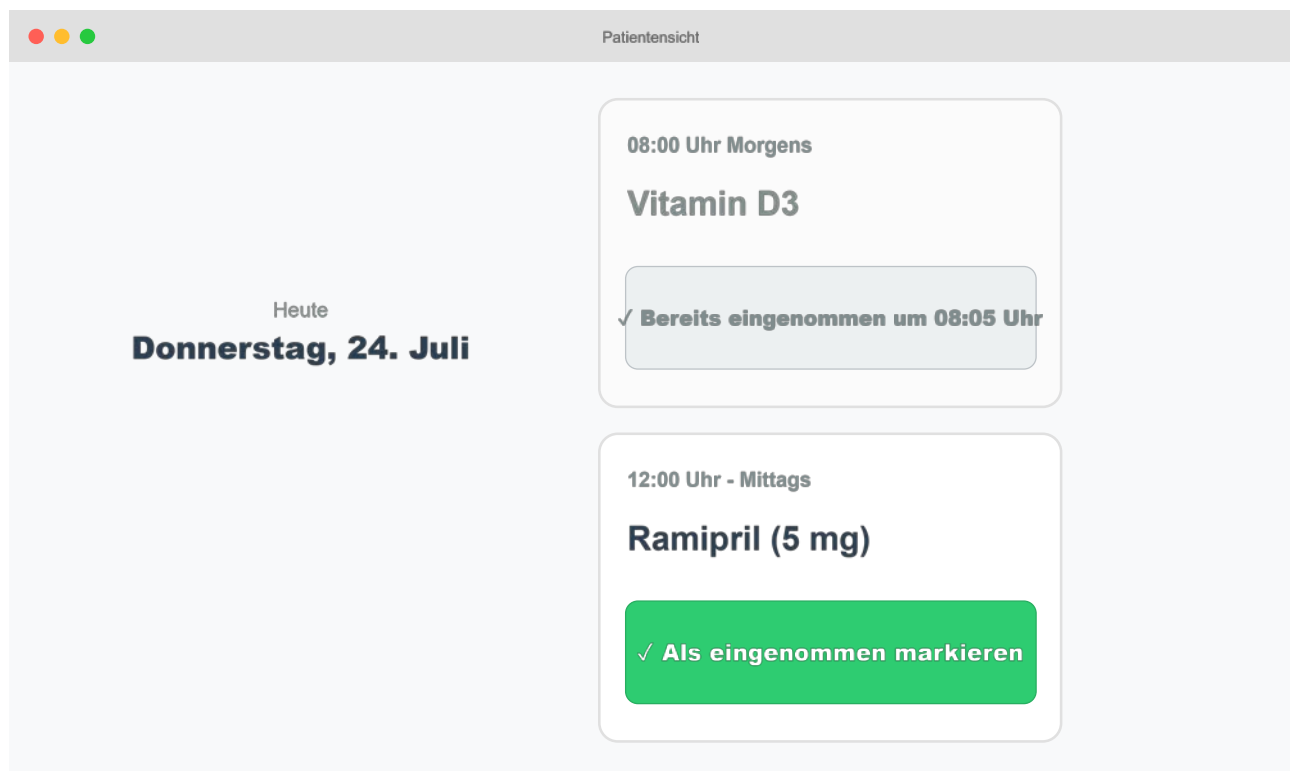
Kognitiv stark reduziertes Single-Page-Design. Verzicht auf störende Navigationsleisten, verschachtelte Menüs und Werbeelemente.

Interaktionselemente & Typografie:

Große, gut lesbare Sans-Serif-Schriftarten mit einer Basis-Schriftgröße von mindestens 18pt. Kontraststarke Farbgestaltung. Großflächige Touch-Buttons zur einfachen Bestätigung der Medikamenteneinnahme.

Die reduzierte und barrierefreie Benutzeroberfläche des Patientenmodus ist in Abbildung 5 illustriert.

Abbildung 5 Patientensicht



Quelle: eigene Darstellung

Wie in Abbildung 5 erkennbar ist, konzentriert sich die Oberfläche ausschließlich auf die zentralen Fragen der zu pflegenden Person („Was muss ich jetzt einnehmen?“). Die Interaktion beschränkt sich auf das einfache Bestätigen über großflächige Schalter, was die Handlungsautonomie des Patienten fördert.

3.4.3 Mehrstufiges Validierungskonzept

Um Fehleingaben, unvollständige Formulare oder schadhafte Manipulationsversuche (z. B. Injection-Angriffe) wirksam abzufangen und die Systemstabilität zu garantieren, setzt die Anwendung auf ein mehrstufiges Validierungskonzept:

Clientseitige Validierung (HTML5 & Browser)

Bereits im Frontend werden Formulareingaben durch native HTML5-Attribute vorab geprüft. Attribute wie `required` (Pflichtfeld), `type="number"` (Erkennen von Falschformaten), `min="0"` (Verhinderung negativer Bestände) sowie `step="0.1"` bieten dem Nutzer eine unmittelbare Rückmeldung noch vor dem Absenden der Daten.

Serverseitige Validierung (Flask & Python)

Da clientseitige Prüfungen im Webbrowser umgangen werden können, führt das Flask-Backend bei jedem eingehenden Request eine erneute Prüfung aller Parameter durch. Datentypen werden explizit konvertiert (z. B. String zu Float) und fachliche Grenzwerte (gemäß den Geschäftsregeln aus Kapitel 3.3) evaluiert.

Fehlerbehandlung und Transaktionssicherheit

Schlägt die Validierung im Backend fehl, wird die angeforderte Datenbankoperation sofort abgebrochen und ein Rollback der Transaktion eingeleitet. Dem Anwender wird das Ursprungsformular mit einer spezifischen, gut verständlichen Fehlermeldung neu gerendert, sodass keine unvollständigen oder fehlerhaften Daten im System verbleiben.

4 Architekturdokumentation

Das vorliegende Kapitel dokumentiert die architektonischen Entscheidungen, die strukturellen Komponenten sowie das dynamische Systemverhalten. Auf Basis der definierten Spezifikationen und Qualitätsanforderungen wird dargelegt, wie die funktionale Trennung, die Datensicherheit und die Barrierefreiheit im Programmcode technisch realisiert wurden.

4.1 Technologieübersicht

Die Auswahl der Basistechnologien erfolgte zielgerichtet anhand der nichtfunktionalen Qualitätskriterien.

Programmiersprache - Python (v3.14+): Python bietet als moderne, objektorientierte Hochsprache eine prägnante Syntax, eine strikte Typisierung sowie ein ausgereiftes Ökosystem für Datenverarbeitung. Die Plattformunabhängigkeit garantiert die native Lauffähigkeit unter Windows 10/11 (NFA-05).

Web-Framework - Flask (v3.1+): Als WSGI-basiertes Micro-Framework zeichnet sich Flask durch einen minimalistischen Kern aus, der die volle Kontrolle über den Request-Response-Lebenszyklus ermöglicht. Der Verzicht auf den monolithischen Overhead größerer Frameworks (wie Django) minimiert die Server-Antwortzeiten (NFA-02) und reduziert potenzielle Angriffsflächen.

Template-Engine - Jinja2: Das serverseitige Rendering erlaubt die dynamische HTML-Generierung bei strikter Trennung von Präsentations- und Geschäftslogik. Das integrierte, kontextsensitive Auto-Escaping verhindert Cross-Site-Scripting-Angriffe (XSS) nativ auf Architekturebene (NFA-03).

CSS-Framework - Bootstrap 5: Die UI-Komponentenbibliothek erlaubt die Umsetzung eines responsiven Gestaltungsrasters. Dies bildet die technische Basis für das duale Oberflächenkonzept: ein informationsdichtes Verwaltungsdashboard für pflegende Angehörige sowie ein kognitiv reduziertes, kontrastreiches Single-Page-Design mit Großtypografie für zu pflegende Personen gemäß WCAG 2.2 (NFA-01).

Persistenzschicht - SQLite & Flask-SQLAlchemy (v3.1+): SQLite operiert als serverlose, prozessintegrierte Relationale Datenbankengine. Die Datenhaltung in einer lokalen Datei (medical_tracker.db) erfüllt das Paradigma der Datensparsamkeit (Privacy by Design nach Art. 25 DSGVO / NFA-03). Flask-SQLAlchemy abstrahiert den SQL-Dialekt über ein objektrelationales Mapping (ORM). Dies unterbindet SQL-Injection-Schwachstellen durch automatisierte Parameter-Bindung und garantiert die zukünftige Portierbarkeit auf Server-Datenbanken (z. B. PostgreSQL nach NFA-07) ohne Code-Refactoring.

Formularvalidierung - Flask-WTF & WTForms (v3.1+): Schützt sämtliche POST-Requests gegen Cross-Site Request Forgery (CSRF) mittels kryptographischer Token und kapselt die mehrstufige serverseitige Plausibilitätsprüfung (GR-01 bis GR-04) vor Erreichen der Datenbankschicht.

Dokumenten-Engine - ReportLab (v4.0+): Ermöglicht die pixelgenaue, vektorbasierte Generierung von DIN-A4-Arztberichten. Durch die In-Memory-Verarbeitung wird die Verweildauer unverschlüsselter Gesundheitsdaten im lokalen Dateisystem auf null reduziert.

Konfigurations- und Umgebungskontrolle - python-dotenv: Ermöglicht die strikte Trennung von Code und Konfiguration, indem sensible Parameter (SECRET_KEY, DATABASE_URL) zur Laufzeit aus einer .env-Datei bezogen werden.

4.2 Architekturübersicht

Das architektonische Gesamtsystem folgt einer hierarchischen Drei-Schichten-Architektur, kombiniert mit dem klassischen MVC.

Schichten und Verantwortlichkeiten

Präsentationsschicht (View): Verantwortlich für das Rendering der Benutzeroberfläche und die Erfassung von Benutzerinteraktionen. Sie besteht aus serverseitig kompilierten Jinja2-Templates, die semantisches HTML5 generieren. Gemäß dem Prinzip der Separation of Concerns (SoC) sind Gestaltungsregeln und dynamisches Verhalten in isolierte Stylesheets (static/css/) und JavaScript-Dateien (static/js/) ausgelagert.

Steuerungs- und Geschäftslogikschicht (Controller / Service): Flask-Blueprints nehmen HTTP-Anfragen entgegen, steuern die Validierungskette über WTForms und führen die Geschäftsregeln aus. Komplexe Domänenlogiken sind in dedizierte Services ausgelagert (Service-Layer-Pattern). Diese Schicht garantiert, dass der Systemzustand konsistent bleibt und unberechtigte oder unplausible Eingaben abgefangen werden.

Datenzugriffs- und Persistenzschicht (Model): Definiert die Entitätsklassen und deren Beziehungen über das SQLAlchemy-ORM. Sie kapselt alle CRUD-Operationen gegen die relationale SQLite-Datenbank und erzwingt Constraints (z. B. Fremdschlüssel, Nicht-Null-Bedingungen).

Tabelle 5 MVC Zuordnungsmatrix

MVC-Komponente	Technologie	Projekt-Artefakte	Verantwortung
Model	Flask-SQLAlchemy, SQLite	app/models/*.py, data- base/medical_tra- cker.db	Persistierung der Ge- schäftsobjekte (Pati- ent, Medikament, Vi- talwert, Termin, Ein- nahmeProtokoll), Fremdschlüssel-Be- ziehungen und DB- Constraints (bestand >= 0).
View	Jinja2, Bootstrap 5, Vanilla JS	app/templa- tes/**/*.html, app/sta- tic/css/, app/static/js/	Aufbereitung und visu- elle Darstellung der UI-Zustände; barriere- freier Patientenmodus (WCAG 2.2 / NFA-01); dynamische Live-Uhr und Eingabemasken- Umschaltung.
Controller	Flask-Blueprints, Flask-WTF	app/routes/*.py, app/forms.py	Entgegennahme der HTTP-Anfragen, CSRF-Schutz, server- seitige Durchsetzung der Geschäftsregeln GR-01 bis GR-04, ato- mare Transaktions- steuerung und Weiter- leitung (PRG-Pattern).
Service (Erweiterung)	ReportLab, Python io	app/services/pdf_ge- nerator.py	Entkoppelte Business- Komponente zur flüch- tigen In-Memory-Kom- pilierung strukturierter DIN-A4-Arztberichte (FA-03 / NFA-03).

Quelle: eigene Darstellung

Zentrale Entwurfsmuster

Application-Factory-Muster (create_app()): Die Flask-Applikationsinstanz wird über eine Fabrikmethode dynamisch instanziiert.

Blueprint-Architektur: Zerlegung der Anwendung in funktionale Module (medikamente_bp, patient_bp, vitaldaten_bp etc.). Dies verhindert monolithische Controller-Dateien und stellt eine hohe Wartbarkeit sicher.

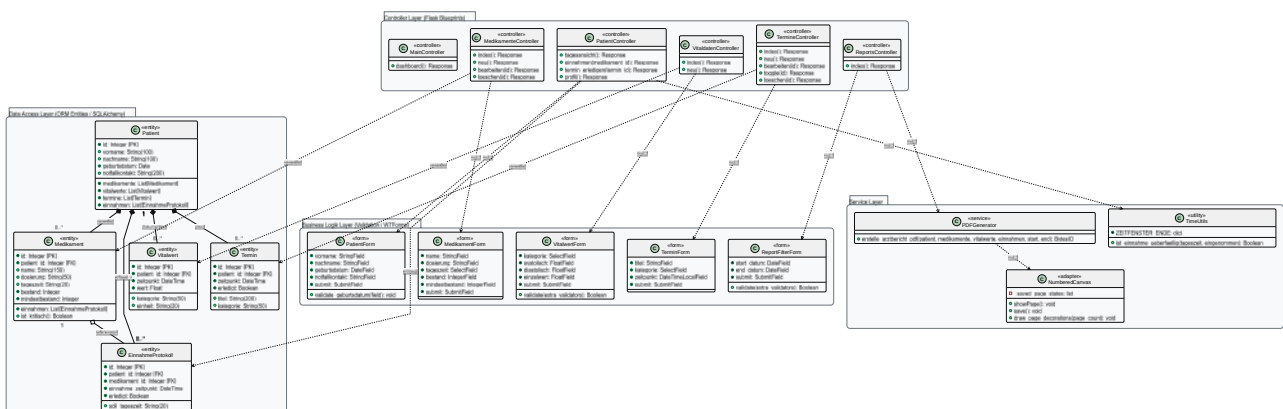
Service-Layer-Muster: Die Auslagerung der PDF-Erstellung in app/services/pdf_generator.py entkoppelt die Präsentations-Controller von der ReportLab-Grafikengine.

Zweistufiger Canvas-Renderer (NumberedCanvas): Ein strukturelles Adaptermuster überschreibt die showPage- und save-Methoden von ReportLab, um die exakte Gesamtseitenzahl zweistufig im Speicher zu akkumulieren und in der Kopf-/Fußzeile auszugeben.

4.3 Komponenten- und Klassenstruktur

Die statische Struktur der Anwendungssoftware gliedert sich in Datenmodelle, Validierungsformulare und funktionale Routencontroller.

Abbildung 6 Detailliertes Klassendiagramm des Gesamtsystems



Quelle: eigene Darstellung

Eine hochauflösende, zoombare Version ist im GitHub Repository verfügbar:

<https://raw.githubusercontent.com/SteffenOe/medicaltracker/refs/heads/main/docs/diagramme/uml-klassendiagramm.drawio.svg>

Persistenzmodelle (Data Layer - app/models/)

Patient: Zentrales Wurzelobjekt des Datenmodells. Attribute: id (Integer, PK), vorname (String(100)), nachname (String(100)), geburtsdatum (Date), notfallkontakt (String(200)).

Beziehungen: Verwaltet 1:n-Assoziationen zu Medikament, Vitalwert, Termin und EinnahmeProtokoll über Fremdschlüssel.

Medikament: Repräsentiert die verordnete Dauermedikation. Attribute: id (Integer, PK), patient_id (Integer, FK), name (String(150)), dosierung (String(50)), tageszeit (String(20)), bestand (Integer), mindestbestand (Integer).

EinnahmeProtokoll: Unveränderliches Dokumentationsobjekt (GR-03). Attribute: id (Integer, PK), patient_id (Integer, FK), medikament_id (Integer, FK), soll_tageszeit (String(50)), einnahme_zeitpunkt (DateTime), erledigt (Boolean).

Vitalwert: Zeitreihendokumentation physiologischer Messwerte. Attribute: id (Integer, PK), patient_id (Integer, FK), zeitpunkt (DateTime), kategorie (String(50)), wert (Float), einheit (String(20)).

Termin: Verwaltung von Pflegeterminen. Attribute: id (Integer, PK), patient_id (Integer, FK), titel (String(200)), kategorie (String(50)), zeitpunkt (DateTime), erledigt (Boolean).

Formular- und Validierungsklassen (Business Logik - app/forms/)

PatientForm: Validiert Eingaben für Personalien und erzwingt über die Methode validate_geburtsdatum() die Geschäftsregel GR-04 ($geburtsdatum < datum_{aktuell}$).

MedikamentForm: Prüft Pflichtangaben und stellt über NumberRange(min=0) sicher, dass Bestände und Schwellenwerte nicht negativ werden (GR-01).

VitalwertForm: Implementiert eine Validierungslogik in validate(): Erzwingt für Blutdruck das physiologische Intervall Blutdruck: $systolisch > diastolisch \wedge (30 \text{ mmHg} \leq diastolisch < systolisch \leq 300 \text{ mmHg})$, für die Pulsfrequenz: $30 \text{ bpm} \leq f_{puls} \leq 250 \text{ bpm}$ und für das Körpergewicht: $2,0 \text{ kg} \leq m \leq 300,0 \text{ kg}$ (GR-02).

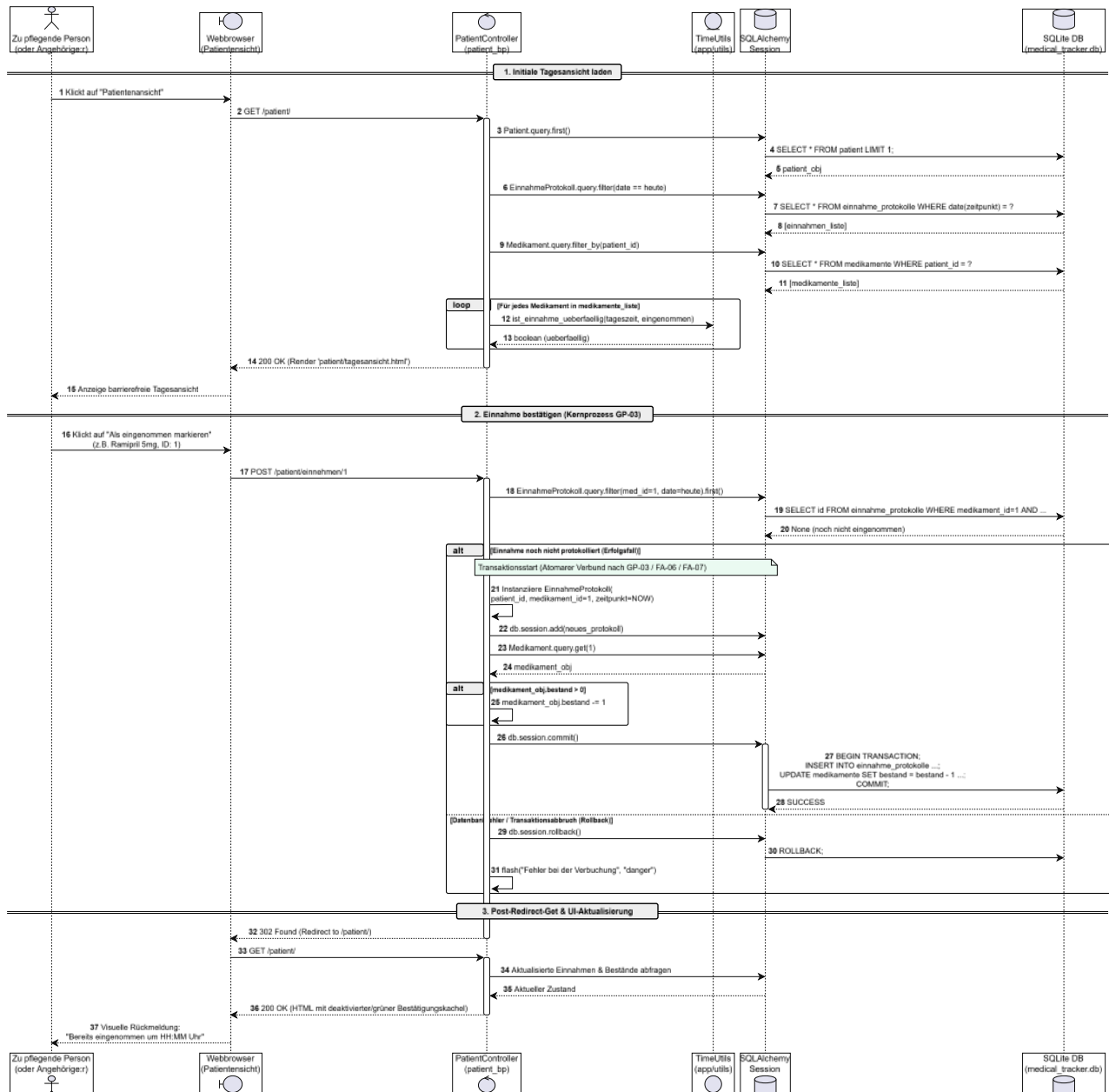
TerminForm: Erfasst über DateTimeLocalField strukturierte Datums- und Uhrzeitwerte für den Terminkalender.

ReportFilterForm: Steuert die Zeitfensterabgrenzung für den PDF-Export und prüft chronologische Konsistenz ($start_datum < end_datum$).

4.4 Dynamisches Systemverhalten (Verhaltenssicht – Kernprozess GP-03)

Der Kernprozess GP-03 (Medikamenteneinnahme und Bestandsaktualisierung) verknüpft die Bestätigung im barrierefreien Patientenmodus mit der transaktionssicheren Fortschreibung des Lagerbestands und der revisionssicheren Protokollerstellung.

Abbildung 7 UML-Sequenzdiagramm des Kernprozesses GP-03



Quelle: eigene Darstellungen

Eine hochauflösende, zoombare Version ist im GitHub Repository verfügbar:

<https://raw.githubusercontent.com/SteffenOe/medicaltracker/refs/heads/main/docs/diagramme/uml-sequenzdiagramm.drawio.svg>

Detaillierter Ablauf des Sequenzdiagramms

- Aufruf der Tagesansicht:** Der Client sendet einen GET /patient/-Request. Der patient_bp-Controller fragt über SQLAlchemy den Patienten sowie die für den heutigen Tag bereits existierenden Einträge aus EinnahmeProtokoll ab.
- Aggregation & Zeitprüfung:** Der Controller iteriert über die verordneten Medikamente, berechnet über das Modul app/utils.py eventuelle Überfälligkeiten anhand der aktuellen

Systemzeit und übergibt die aggregierte Datenstruktur an das Template `patient/tagesansicht.html`.

3. Auslösen der Einnahmebestätigung: Die anwendende Person betätigt die großflächige Schaltfläche „Als eingenommen markieren“. Der Browser übermittelt einen POST `/patient/einnehmen/<id>`-Request an das Backend.
4. Transaktionskapselung & Duplikatsprüfung: Der Controller prüft, ob für die Kombination aus `medikament_id` und aktuellem Datum bereits ein Protokolleintrag vorliegt (Schutz vor versehentlichen doppelter Einnahme). Falls unbestätigt, instanziiert das System ein neues `EinnahmeProtokoll`-Objekt mit aktuellem Zeitstempel (`datetime.now()`). Zeitgleich wird das betroffene `Medikament`-Objekt geladen und dessen Attribut `bestand` dekrementiert (*`bestand = bestand - 1`*, GR-01).
5. Atomarer Commit: Die Operationen werden über `db.session.commit()` atomar in die SQLite-Datenbank geschrieben. Tritt währenddessen ein Systemfehler auf, fängt ein `except`-Block die Exception ab, stößt ein `db.session.rollback()` an und setzt eine Flash-Fehlermeldung.
6. PRG-Muster (Post-Redirect-Get): Nach erfolgreichem Commit leitet der Server via HTTP 302 Redirect auf `/patient/` um. Dies verhindert, dass ein versehentliches Neuladen der Seite im Browser den Einnahme-Request erneut absendet. Die Benutzeroberfläche aktualisiert sich und rendert die betroffene Dosis fortan als schreibgeschützte, grüne Bestätigungsbox.

Literaturverzeichnis

Deutsches Institut für Normung. (2015). *Metrologische Aspekte nichtselbsttätiger Waagen* (DIN EN 45501:2015-04).

Deutsches Institut für Normung. (2019). *Medizinische elektrische Geräte — Teil 2-61: Besondere Festlegungen für die Sicherheit einschließlich der wesentlichen Leistungsmerkmale von Pulsoxi-metriegeräten* (DIN EN ISO 80601-2-61:2019-09).

Deutsches Institut für Normung. (2024). *Nichtinvasive Blutdruckmessgeräte - Teil 2: Klinische Prü-fung der intermittierenden automatisierten Bauart* (DIN EN ISO 81060-2:2024-08)

Glikson, M., Nielsen, J. C., Kronborg, M. B., Michowitz, Y., Auricchio, A., Barbash, I. M., Barrabés, J. A., Boriani, G., Braunschweig, F., Brignole, M., Burri, H., Coats, A. J. S., Deharo, J.-C., Delgado, V., Diller, G.-P., Israel, C. W., Keren, A., Knops, R. E., Kotecha, D., . . . Madhavan, M. (2021). 2021 ESC Guidelines on cardiac pacing and cardiac resynchronization therapy. *European Heart Journal*, 42(35), 3427–3520. <https://doi.org/10.1093/eurheartj/ehab364>

International Organization for Standardization. (2023). *Systems and software engineering — Sys-tems and software quality models* (ISO/IEC Standard No. 25010:2023).

McEvoy, J. W., McCarthy, C. P., Bruno, R. M., Brouwers, S., Canavan, M. D., Ceconi, C., Christo-dorescu, R. M., Daskalopoulou, S. S., Ferro, C. J., Gerdts, E., Hanssen, H., Harris, J., Lauder, L., McManus, R. J., Molloy, G. J., Rahimi, K., Regitz-Zagrosek, V., Rossi, G. P., Sandset, E. C., . . . Khamidullaeva, G. A. (2024). 2024 ESC Guidelines for the management of elevated blood pressure and hypertension. *European Heart Journal*, 45(38), 3912–4018. <https://doi.org/10.1093/eurhe-arti/ehae178>

Pohl, K. & Rupp, C. (2021). *Basiswissen Requirements Engineering: Aus- und Weiterbildung nach IREB-Standard zum Certified Professional for Requirements Engineering Foundation Level*.

W3C. (2023). *Web content accessibility guidelines (WCAG) 2.2* (W3C Recommendation). W3C.